

MODULE 09 · DOMAIN 2 - IMPLEMENTATION AND INTEGRATION

Deployment Strategies

Task 2.2. Bedrock against SageMaker AI, endpoint types, GPU and memory realities, and model cascading.

DOMAIN 2 of 5 EXAM WEIGHT 26% MODULE 9 of 22

On Bedrock you never see a GPU. So the real deployment question is narrower than it looks: **when do you not use Bedrock?**

Bedrock or SageMaker AI hosting?

AMAZON BEDROCK

Serverless, per token

- No instances, no GPUs, no scaling policy
- Guardrails, Knowledge Bases, Flows built in
- The model catalogue AWS offers
- Pay per token, or reserve capacity

SAGEMAKER AI

Your model, your instances

- Any open-weight model, including your own
- Full control of container, GPU and runtime
- Quantisation, compilation, custom serving stack
- Pay per instance hour, running or idle

THE TIEBREAKER

Four things force SageMaker: a model Bedrock does not host, weights you cannot upload, a serving optimisation you must control, or hardware requirements you must specify.

Four routes to a hosted open-weight model

ROUTE	WHAT IT GIVES YOU	NOTE
SageMaker JumpStart	One-click deploy of open models	Prebuilt containers and recipes
Bedrock Marketplace	Third-party models inside Bedrock	Endpoint you manage
Custom Model Import	Your weights, open architectures	Invoked like any Bedrock model
Bring your own container	Full control of the serving stack	You own every bug in it

ON THE EXAM

JumpStart is the fastest path to a working endpoint. Custom Model Import is the route that keeps you inside Bedrock's API, guardrails and knowledge bases.

The four SageMaker inference options

OPTION	LIMITS	CHOOSE IT WHEN
Real-time	6 MB payload, 60s timeout, GPU	A caller waits. Steady, predictable traffic
Serverless	4 MB payload, 60s, scales to zero	Intermittent traffic that tolerates cold starts
Asynchronous	1 GB payload, up to 1 hour, scales to zero	Large inputs or long processing, near-real-time
Batch transform	Whole dataset, instances stop after	Offline scoring of data you already have

ON THE EXAM

Two of these scale to zero. Asynchronous inference is the one people forget, and it is the only option combining a 1 GB payload, an hour of processing and zero cost when idle.

How the numbers pick the option

- ◆ **Payload over 6 MB** eliminates real-time immediately. Video, long audio, large documents.
- ◆ **Processing over 60 seconds** eliminates real-time and serverless together.
- ◆ **"Idle periods between traffic spurts"** is the exam's phrase for serverless.
- ◆ **"Results within minutes"** plus large inputs is asynchronous, not batch.
- ◆ **Cold starts** are the serverless trade. A large model loading on a cold invocation is not a few hundred milliseconds.

ON THE EXAM

Batch transform and asynchronous inference differ on arrival: batch processes a dataset you already have, asynchronous processes requests as they arrive but without a waiting caller.

A media company self-hosts an open-weight speech model on SageMaker AI to transcribe and summarise recorded interviews. Files average 400 MB and take four to twelve minutes to process.

Journalists upload files sporadically through the working day, with long quiet periods overnight and at weekends. They accept waiting several minutes for a result, but expect it without having to reschedule anything. The current real-time endpoint runs continuously and frequently returns errors on larger files.

Which deployment option should the company use?

- | | |
|--|--|
| ☐ A Real-time endpoint with a larger instance type and an increased timeout | ☐ B Asynchronous inference, with the payload in Amazon S3 and autoscaling to zero when idle |
| ☐ C Serverless inference, so the endpoint scales to zero overnight | ☐ D Batch transform, run on a schedule every fifteen minutes |

◆ PAUSE AND DECIDE

Which deployment option should the company use?

- A

 Bigger real-time instance, longer timeout
The 6 MB payload cap and 60-second timeout are option limits, not instance limits. A larger instance changes neither.
- C

 Serverless inference
Scales to zero as required, and caps at a 4 MB payload and 60 seconds. Both limits are breached before the file finishes uploading.
- B

 Asynchronous inference with S3 payloads
1 GB payloads and up to an hour of processing cover 400 MB files and twelve minutes, and it scales to zero through the quiet periods.
- D

 Scheduled batch transform
Would process the files, and makes results arrive on a timetable rather than on submission. The scenario rules out rescheduling.

THE SIGNAL

Three numbers and one phrase: 400 MB, twelve minutes, long idle periods, and results "without having to reschedule anything". Only asynchronous inference satisfies all four.

A traditional model's memory is its weights.
An LLM's memory is its weights **plus a KV cache that grows with every token and every concurrent user.**

Where the GPU memory goes

- ◆ **Weights.** Roughly two bytes per parameter at 16-bit precision. A 70-billion-parameter model needs about 140 GB.
- ◆ **KV cache.** Scales with layers, hidden size, sequence length and concurrency. It is the variable part.
- ◆ **Quantisation** shrinks the weights. FP8 roughly halves them against 16-bit and frees room for a bigger cache.
- ◆ FP8 support is hardware-dependent. Newer instances quantise weights *and* activations; older ones only weights.
- ◆ Out-of-memory failures under load are a **KV cache** problem far more often than a weights problem.

ON THE EXAM

This is why token processing capacity appears in the exam guide alongside memory. On an LLM endpoint, tokens in flight are what consume the hardware.

Loading a very large model quickly

- ◆ The naive path is slow. Download from S3 to disk, load into host memory, shard, then push to GPUs.
- ◆ **Fast model loading** pre-shards the model and streams weights straight from S3 onto the GPUs, skipping those steps.
- ◆ **Ahead-of-time compilation** with TensorRT-LLM removes just-in-time compilation from every new instance.
- ◆ SageMaker supports models up to **500 GB**, with health check and download timeouts configurable up to **60 minutes**.
- ◆ Loading time *is* scaling latency. A model taking eight minutes to load cannot scale out to meet a spike.

ON THE EXAM

That last point is the one the exam tests. Autoscaling only helps if a new instance becomes useful before the spike ends, and for large models that is not automatic.

A company hosts a large open-weight model on SageMaker AI real-time endpoints with a target-tracking autoscaling policy. Traffic triples every weekday at 08:00.

During the ramp, latency degrades badly for around nine minutes before recovering. Metrics show autoscaling triggering promptly and new instances taking about eight minutes to begin serving. GPU utilisation on running instances is near saturation throughout. The team has proposed lowering the autoscaling threshold so it triggers sooner.

What is the most effective change?

- ☐ A Lower the autoscaling target so scaling triggers earlier
- ☐ B Apply fast model loading and ahead-of-time compilation to cut instance start-up time, and add a scheduled scaling action before 08:00
- ☐ C Move the endpoint to serverless inference so capacity appears instantly
- ☐ D Increase the instance size so fewer instances are needed

◆ PAUSE AND DECIDE

What is the most effective change?

A Lower the autoscaling target

Scaling already triggers promptly. Triggering earlier still waits eight minutes for a usable instance, so it moves the gap rather than closing it.

C Serverless inference

Capacity does not appear instantly for a large model. Serverless has cold starts too, plus a 4 MB and 60-second ceiling.

B Faster loading plus scheduled scaling

Attacks the eight minutes directly, and scheduled scaling adds capacity before a spike the team can predict to the minute.

D Larger instances

Fewer, larger instances take at least as long to load. It reduces how often you scale without reducing how long scaling takes.

THE SIGNAL

"Autoscaling triggering promptly" and "eight minutes to begin serving" together locate the fault. The delay is model load time, and a predictable 08:00 spike does not need reactive scaling at all.

Single-model endpoints or inference components?

SINGLE-MODEL ENDPOINT

One model, one endpoint

- Simplest to reason about and monitor
- Independent scaling per model
- Each endpoint pays for its own instances
- Idle GPUs multiply across models

INFERENCE COMPONENTS

Several models, one endpoint

- Models share instances and GPUs
- Allocate compute per component
- Scale components independently
- One noisy model can affect neighbours

THE TIEBREAKER

The signal is idle GPU cost across several low-traffic models. Consolidating them onto shared instances is what inference components exist to do.

A research organisation hosts seven fine-tuned open-weight models, each on its own SageMaker AI real-time endpoint, each on a single GPU instance. Six of them serve fewer than 200 requests a day. The seventh serves steady production traffic.

The GPU bill is dominated by idle capacity on the six quiet endpoints, which cannot be shut down because researchers query them unpredictably. Response times of a few seconds are acceptable for the six. The production endpoint's performance must not be affected.

What should the organisation do?

- | | |
|--|--|
| <p>A Consolidate the six low-traffic models onto one endpoint using inference components, leaving the production model on its own endpoint</p> | <p>B Consolidate all seven models onto one endpoint using inference components</p> |
| <p>C Move all seven models to serverless inference endpoints</p> | <p>D Move the six low-traffic models to asynchronous inference endpoints</p> |

◆ PAUSE AND DECIDE

What should the organisation do?

- A

 Consolidate the six, leave production alone
Six idle GPUs become one shared endpoint, and the isolation requirement for the production model is honoured by not touching it.
- B

 Consolidate all seven
Cheapest, and it puts production on shared instances with six unpredictable neighbours. The scenario forbids affecting it.
- C

 All seven to serverless
Serverless has no GPU support for this class of model, and a 60-second processing ceiling. It does not host large open-weight models.
- D

 The six to asynchronous inference
Would scale to zero and help, at the cost of turning every researcher query into a submit-and-collect flow for a few seconds of work.

THE SIGNAL

Two requirements pull opposite ways: consolidate for cost, isolate production for performance. The answer that splits the estate honours both; the cheapest answer honours only one.

The cheapest deployment is the one you do not make. Most requests hitting a frontier model **did not need a frontier model.**

Model cascading

01

Try cheap

The small model handles the request first.

02

Check

Score confidence, schema validity, or a rule.

03

Accept

Good enough? Return it. Most requests stop here.

04

Escalate

Otherwise, re-run on the larger model.

ON THE EXAM

The economics only work if the check is cheap and the escalation rate is low. If half of requests escalate, you have paid for both models on half your traffic.

Cascading or intelligent prompt routing?

CASCADING

Try, check, escalate

- You write the check and the threshold
- Decision made after seeing an answer
- Escalated requests pay twice
- Works with any two models, any providers

INTELLIGENT PROMPT ROUTING

Predict, then route once

- Bedrock predicts difficulty before invoking
- Each request hits exactly one model
- Managed, one parameter, no check to write
- Two models from the same family

THE TIEBREAKER

Routing predicts from the prompt and never pays twice. Cascading judges the actual answer and can, which makes it more accurate and more expensive when it is wrong.

A support platform classifies incoming tickets into one of twelve categories using a frontier model on Bedrock. Analysis shows 88% of tickets are unambiguous and a small model classifies them identically to the frontier model. The remaining 12% are genuinely ambiguous. The classification is returned as structured JSON with a category and a confidence value. Cost must fall substantially without any measurable drop in classification accuracy.

Which approach best meets the requirement?

- ☐ A Switch entirely to the small model and accept the accuracy loss on ambiguous tickets
- ☐ B Cascade: classify with the small model, and re-run on the frontier model only when the returned confidence falls below a tuned threshold
- ☐ C Use Bedrock Intelligent Prompt Routing between the two models
- ☐ D Keep the frontier model and enable prompt caching on the classification instructions

◆ PAUSE AND DECIDE

Which approach best meets the requirement?

A Small model only

Explicitly violates the no-measurable-accuracy-drop constraint on the 12% that are genuinely ambiguous.

C Intelligent Prompt Routing

Predicts difficulty from the prompt before seeing an answer. Ticket ambiguity is often only apparent once you attempt the classification.

B Cascade on returned confidence

The output already carries a confidence value, so the check is free. 88% of traffic stops at the small model, and the hard 12% still reaches the frontier model.

D Prompt caching on instructions

Caches a short instruction block. Every ticket still runs through the frontier model, so the dominant cost is untouched.

THE SIGNAL

"Returned as structured JSON with a confidence value" is the scenario handing you a free cascade check. Cascading is expensive when you must build the judge; here it already exists.

The hybrid architecture the exam guide names

- ◆ **Bedrock for the general path**, a SageMaker endpoint for the one specialist or fine-tuned model.
- ◆ Lambda routes between them. Neither the caller nor the prompt logic needs to know which served the request.
- ◆ Common shape: Bedrock answers most queries, a domain model handles a narrow high-value task.
- ◆ You keep Bedrock's guardrails, knowledge bases and per-token pricing everywhere except where you genuinely need control.
- ◆ The cost is two serving paths to monitor, secure and keep current.

ON THE EXAM

Skill 2.2.1 names Lambda, Bedrock provisioned throughput and SageMaker endpoints in one sentence, describing hybrid solutions. This is that sentence drawn as an architecture.

Where the application layer runs

- ◆ **Lambda** for request-driven work under fifteen minutes. The default for calling Bedrock.
- ◆ **Fargate** when you need a container without managing nodes. Long-running, larger dependencies.
- ◆ **ECS or EKS** when you need node control, GPUs, or an existing orchestration standard.
- ◆ **App Runner** for a simple containerised web service with the least configuration.
- ◆ This layer *calls* the model. Do not confuse choosing it with choosing where the model is hosted.

ON THE EXAM

Skill 2.1.7 already used this split for MCP servers. The same reasoning applies to any component around the model: duration, state, dependencies and who operates it.

A startup wants to deploy an open-weight 70-billion-parameter model on SageMaker AI, arguing that self-hosting will be cheaper than paying Bedrock per token. Expected traffic is around 8,000 requests a day, spread unevenly.

The team has no MLOps engineers. The same model family is available on Bedrock. They have not yet benchmarked cost, and have not accounted for GPU instance hours during idle periods, model loading time, or scaling behaviour.

What should the team do?

- A Proceed with self-hosting on SageMaker AI, using inference components to share GPU capacity
- B Use the model on Bedrock, and revisit self-hosting only if measured per-token spend exceeds the fully loaded cost of running and operating GPU instances
- C Self-host on Amazon EKS with GPU nodes for maximum control over cost
- D Self-host using SageMaker serverless inference so there is no idle cost

◆ PAUSE AND DECIDE

What should the team do?

- A

 Self-host with inference components
Sharing GPUs helps when you have several models. With one model and uneven traffic, the idle GPU hours remain the dominant cost.
- B

 Bedrock now, measure, revisit later
8,000 uneven requests a day will not saturate a GPU fleet, and a team with no MLOps engineers would be buying an operational burden to avoid an unmeasured bill.
- C

 Self-host on EKS
Maximum control and the highest operational cost of the options, for a team that has nobody to operate it.
- D

 SageMaker serverless inference
Does not support GPU-backed hosting of a model this size, so it cannot run this workload at all.

THE SIGNAL

Two facts settle it: 8,000 uneven daily requests will not keep a GPU busy, and "no MLOps engineers" is a stated constraint. The premise that self-hosting is cheaper was never measured.

The five things to carry forward

- ◆ **Bedrock is the default.** Self-host for a model it does not offer, weights you own, or control you need.
- ◆ **Learn the four endpoint limits.** 6 MB and 60s, 4 MB and 60s, 1 GB and an hour, or a whole dataset offline.
- ◆ **LLM memory is weights plus KV cache.** Out-of-memory under load is usually the cache.
- ◆ **Loading time is scaling latency.** Fast model loading, AOT compilation, and scheduled scaling for predictable spikes.
- ◆ **Cascade when the check is cheap.** Route when difficulty is predictable from the prompt.

ON THE EXAM

Module 10 closes Domain 2 with enterprise integration: event-driven architectures, identity federation, data residency, CI/CD and the GenAI gateway.